



Aarti Kushwaha

Academic Session 2025–2026

1. Introduction & Background Study

What is Data Compression?

- Process of reducing physical storage footprint and transmission bandwidth of digital files.
- **Lossless Compression:** Guarantees 100% exact bit-for-bit reconstruction of original data without distortion or data loss.

Theoretical Foundation

- **Shannon Entropy (1948):** Theoretical minimum limit for average bits per symbol:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

- **Huffman Algorithm (1952):** Greedy prefix-coding algorithm that assigns shorter codes to frequent symbols, achieving optimal prefix compression.

2. Problem Statement & Objectives

Problem Statement

- Growing volumes of digital text and binary records strain storage and network bandwidth.
- Existing industrial utilities are complex, proprietary, or act as closed “black-box” programs.
- Need for a transparent, efficient, self-contained implementation of Huffman entropy coding.

Key Project Objectives

- ① Develop pure lossless Huffman compression and decompression engine.
- ② Build min-heap priority queue for optimal prefix tree construction.
- ③ Implement sequential bit-level bitstream packing.
- ④ Guarantee data integrity via pre/post SHA-256 hash checks.
- ⑤ Deliver an intuitive web-based interface for user interaction.

3. Literature Review & Existing System Analysis

Literature Benchmark

- **C. E. Shannon (1948)**: Proved limits on lossless data compression and statistical redundancy.
- **D. A. Huffman (1952)**: Established bottom-up greedy binary tree synthesis for minimum-redundancy prefix codes.
- **D. E. Knuth (1985)**: Researched efficient data structures for tree construction and bitstream serialization.

Comparison: Existing vs. Proposed System

Feature	Traditional Utilities	Proposed System
Complexity	Multi-layer / Proprietary	Clean Huffman Model
Transparency	Opaque Black-Box	Clear Step-by-Step Execution
Lossless Integrity	Standard	Cryptographically Verified (SHA-256)
User Accessibility	CLI / Heavy Desktop App	Lightweight Web Dashboard

4. Methodology & Working Mechanism

Five Steps of Huffman Compression

- ➊ **Frequency Profiling:** Count occurrences of all 256 byte values (0x00 to 0xFF).
- ➋ **Min-Heap Initialization:** Insert character nodes ordered by frequency.
- ➌ **Tree Construction:** Repeatedly merge lowest two nodes until one root remains.
- ➍ **Code Assignment:** Left branch = '0', Right branch = '1'; extract variable-length bitstrings.
- ➎ **Bitstream Packing:** Pack bit sequences into 8-bit bytes with header frequency metadata.

Decompression Workflow

- Read frequency table → Reconstruct exact binary tree → Traverse bits from root to leaf → Output original bytes → Verify SHA-256 checksum.

5. Project Timeline (4-Month Gantt Chart)

Implementation Schedule

Project Phase / Milestone	Month 1	Month 2	Month 3	Month 4
Requirement Gathering & Literature Review	[XXXX]			
Feasibility Study & System Architecture	[XXXX]			
Huffman Min-Heap Engine Implementation		[XXXX]		
Bit Packing & Serialization Module		[XXXX]		
Web Application Interface & Ingestion API			[XXXX]	
Session Management & File History			[XXXX]	
Testing, Lossless Verification & Benchmarking				[XXXX]
Final Documentation, Presentation & Defense				[XXXX]

Software Requirements: PHP 8.2+, SQLite/MySQL, Modern Browser.

Hardware Requirements: 2.0+ GHz CPU, 4 GB RAM, 500 MB Storage.

6. Expected Outcomes & Conclusion

Expected Outcomes

- Functional, high-performance Huffman compression and decompression tool.
- 20% to 60% space savings on uncompressed text, logs, and source code.
- 100% verified lossless reconstruction with zero byte discrepancy.
- Simple, intuitive web interface accessible from any browser.

Thank You!

Questions & Answers

Candidate: Aarti Kushwaha / Academic Session 2025–2026